

Lab 5, Act 2: Answer Key

When Calculus Runs Out

NRES 746, Instructor Answer Key (not for student distribution)

Each item repeats the handout's question, then gives the answer, a short rationale, and mistakes to watch for. Questions with one correct answer are answered directly. Open-ended items (1c, 1e, the judgment parts of 2b and 2d, 2e, the early-finisher question, and the reflection) get sample answers or discussion points instead. All numbers are computed in R when this file is knitted.

Because students may keep or drop the $\log(y_i!)$ constant, NLL values are given both ways where it matters. The constant is $\sum \log(y_i!) = 155.14$ for these data. Differences between NLLs are the same either way.

Model: $y_i \sim \text{Poisson}(\lambda_i)$, with $\lambda_i = a(1 - e^{-bd_i})$.

Part 1: Plan it on paper

1a.

What do a and b mean biologically, and what are their units?

Answer. a is the expected number of calling males heard far from the road, where the chorus has fully recovered (units: males per 5-minute count). b is how quickly the chorus recovers with distance (units: 1/km). Larger b means faster recovery.

Why. As $d \rightarrow \infty$, $e^{-bd} \rightarrow 0$ and $\lambda \rightarrow a$. The exponent bd must be unitless, so b has units of 1/distance. A useful translation: the chorus reaches half its full size at $d = \ln 2/b$.

Watch for.

- Calling a “the count at the road.” At $d = 0$ the model predicts zero, not a .
- Giving b the same units as d (km).

1b.

Act 1 used a binomial distribution. Why is a Poisson distribution a more reasonable choice for these counts?

Answer. The counts have no fixed number of trials or upper limit. The binomial needs a known N (Act 1 had 10 visits). Here we don't know how many males are present, only how many we heard.

Why. The Poisson describes counts of independent events with no upper bound, with mean (and variance) λ .

Watch for. “Because the counts are bigger” or “because the data are counts.” Both distributions model counts. The key difference is whether there is a known number of trials.

1c.

Use the figure below and your function to estimate starting values for a and b . Show how you got your estimated starting value for b . *Hint for b :* from the figure, estimate the distance $d_{1/2}$ at which the chorus is halfway to its maximum, so that $\lambda(d_{1/2}) = a/2$. Then solve that equation for b . [...]

Answer (any reasonable values). $a \approx 12$ (where the points level off, roughly 11 to 13). For b : the curve reaches half of a (about 6) at roughly $d \approx 0.7$ to 1 km. Setting $a(1 - e^{-bd}) = a/2$ gives $e^{-bd} = 1/2$, so $b = \ln 2/d \approx 0.7$ to 1.

Why. Starting values only need to be in the right neighborhood. Every start tried in 2b (a from 10 to 15, b from 0.2 to 2) reaches the same answer.

Watch for.

- Stopping at $d_{1/2}$ and reporting it as b , e.g. “ $b \approx 1$ km.” It’s a rate, so it needs the $\ln 2/d_{1/2}$ step.
- Algebra slips solving $1 - e^{-bd} = 1/2$: taking the log of both sides before isolating e^{-bd} , or losing the minus sign ($e^{-bd} = 1/2 \Rightarrow -bd = \ln(1/2) = -\ln 2$).
- Setting a to the single highest point (14) rather than the level where the points cluster. Acceptable as a start, but worth a comment.

1d.

Write the negative log-likelihood, $\text{NLL}(a, b)$, for the 15 ponds using the Poisson pmf. Simplify it as far as you can. As in Act 1 (1c), you can drop any term that doesn’t depend on a or b , since it won’t change where the minimum is.

Answer.

$$\text{NLL}(a, b) = - \sum_{i=1}^{15} [y_i \log \lambda_i - \lambda_i - \log(y_i!)] \longrightarrow \sum_{i=1}^{15} [\lambda_i - y_i \log \lambda_i]$$

with $\lambda_i = a(1 - e^{-bd_i})$. Fully expanded:

$$\text{NLL}(a, b) = \sum_{i=1}^{15} [a(1 - e^{-bd_i}) - y_i \log a - y_i \log(1 - e^{-bd_i})]$$

Why. The Poisson pmf is $\lambda^y e^{-\lambda}/y!$. Taking the log gives $y \log \lambda - \lambda - \log y!$. Negate and sum. The $\log(y_i!)$ term doesn’t involve a or b , so it can be dropped.

Watch for.

- Sign errors when negating (the λ_i term should end up positive).
- Writing $\log \lambda_i$ as $\log a \cdot \log(1 - e^{-bd_i})$ instead of a sum.
- Using a single λ for all ponds, forgetting the subscript i .

1e.

In plain English (not R code), describe step by step what a function must do to compute the NLL for one candidate pair (a, b) . Then describe how you would use that function to find the MLE.

Sample answer. Given a candidate a and b : (1) for each pond, compute the expected count λ_i from its distance using the monomolecular formula; (2) for each pond, compute the log of the Poisson probability of the observed count given λ_i ; (3) add these up across ponds; (4) return the negative of the sum. To find the MLE, give this function and a starting guess (from 1c) to an optimizer, which tries new (a, b) pairs, moving toward lower NLL, until it can’t improve. A grid search over (a, b) also works.

Watch for.

- Plans that compute one λ for all ponds.
- Plans that maximize the NLL, or that forget the minus sign.
- R syntax instead of plain English (the handout asks for English).

Part 2: Fit the model

2a.

Write a function `nll_mono(par, y, d)` that implements your NLL from 1d, where `par` holds a and b . Check it: `nll_mono(c(10, 1), males, dist_km)` should return about 33.84 if you used `dpois()`, or about -121.31 if you coded the formula yourself and dropped the constant term. (Why can the second number be negative?)

Answer.

```
dist_km <- chorus_data$dist_km
males <- chorus_data$males

nll_mono <- function(par, y, d) {
  lambda <- par[1] * (1 - exp(-par[2] * d))
  -sum(dpois(y, lambda = lambda, log = TRUE))
}

nll_mono_nc <- function(par, y, d) {
  lambda <- par[1] * (1 - exp(-par[2] * d))
  sum(lambda - y * log(lambda))
}

nll_mono(c(10, 1), males, dist_km)

## [1] 33.83862
nll_mono_nc(c(10, 1), males, dist_km)
```

```
## [1] -121.3056
```

Why negative? Dropping the constant subtracts $\sum \log(y_i!) = 155.14$ from every NLL value. What's left is no longer the negative log of a probability, so nothing keeps it above zero. Only differences between NLL values matter.

Watch for.

- `dpois(lambda, y)` with the arguments swapped.
- `par$a` on an unnamed vector (returns an error or NULL).
- Forgetting the minus sign in front of `sum()`.

2b.

Evaluate the NLL at your starting values from 1c. Then use `optim()`, starting from those values, to find the MLE. [...] Did `optim()` print any warnings? If so, what do you think caused them, and do they mean the answer is wrong?

Answer.

```
start <- c(12, 0.8)
nll_mono(start, males, dist_km)

## [1] 33.51511

fit <- optim(start, nll_mono, y = males, d = dist_km)
fit$par

## [1] 12.0013633 0.6373799

fit$value

## [1] 32.96371

fit$convergence
```

```
## [1] 0
```

$\hat{a} \approx 12$, $\hat{b} \approx 0.64$, NLL at the MLE ≈ 32.96 (or -122.18 without the constant), convergence code 0.

Other reasonable starts land on the same answer:

```
starts <- list(c(10, 1), c(14, 0.5), c(15, 2), c(12, 0.2))
t(sapply(starts, function(s) round(optim(s, nll_mono, y = males, d = dist_km)$par, 3)))
```

```
##           [,1]  [,2]
## [1,] 12.002 0.637
## [2,] 12.000 0.638
## [3,] 12.000 0.637
## [4,] 12.000 0.638
```

Warnings (discussion). Most starting values produce “NaNs produced” warnings. While searching, `optim()` tries some negative values of a or b , which make λ_i negative (or zero), and `dpois()` returns NaN there. The warnings do **not** mean the final answer is wrong: convergence code 0 and estimates that match the plot say the search recovered. Two standard fixes, worth mentioning to students who ask: bounds (`method = "L-BFGS-B"` with `lower`), or fitting $\log a$ and $\log b$ so the parameters can’t go negative.

Watch for.

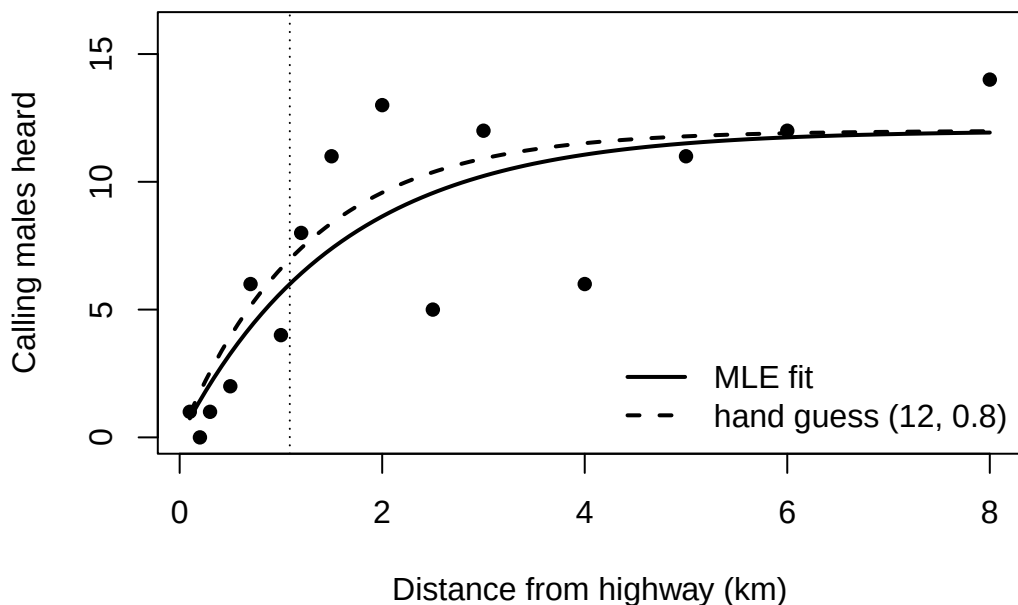
- Ignoring the convergence code.
- Concluding the fit failed because of the warnings.
- Starting values far off (e.g. $b = 50$) that send `optim()` to a flat region. If a student gets a strange answer, check their start.

2c.

Plot the data with your fitted curve and your hand-guessed curve from 1c. How far from the road does the expected chorus reach half its full size?

Answer. $\ln 2 / \hat{b} \approx 1.09$ km.

```
par(mar = c(4, 4.5, 0.5, 0.5))
plot(dist_km, males, pch = 16, xlab = "Distance from highway (km)",
     ylab = "Calling males heard", ylim = c(0, 16))
curve(fit$par[1] * (1 - exp(-fit$par[2] * x)), add = TRUE, lwd = 2)
curve(start[1] * (1 - exp(-start[2] * x)), add = TRUE, lwd = 2, lty = 2)
abline(v = log(2) / fit$par[2], lty = 3)
legend("bottomright", c("MLE fit", "hand guess (12, 0.8)"),
     lwd = 2, lty = c(1, 2), bty = "n")
```



Why. Half of a means $1 - e^{-bd} = 1/2$, so $d = \ln 2/b$ (the same step as 1c, now with $\hat{b}$).

Watch for.

- Reporting $1/\hat{b}$ (about 1.57 km). That's where the chorus reaches $1 - e^{-1} \approx 63\%$ of full size, not half.
- Reading the half-size distance off the raw data (e.g. the first pond with 6 or more males) instead of the fitted curve.

2d.

A skeptic says the road has no effect: every pond has the same expected count λ , whatever its distance. What is the MLE of λ under that model, and what is its NLL? (Keep or drop the constant term the same way you did in 2a, so the two NLLs are comparable.) Which model do the data support more? Does the size of the difference convince you?

Answer.

```
lambda_hat <- mean(males)
nll_null <- -sum(dpois(males, lambda_hat, log = TRUE))
c(lambda_hat = lambda_hat, nll_null = nll_null,
  nll_null_nc = nll_null - log_fact, difference = nll_null - fit$value)
```

```
## lambda_hat    nll_null nll_null_nc difference
##      7.066667    53.873005 -101.271223    20.909293
```

$\hat{\lambda} = \bar{y} = 106/15 \approx 7.07$. NLL ≈ 53.87 (or -101.27 without the constant). The road model's NLL is lower by about 20.9 units, so the data support it much more.

Why (discussion). The Poisson MLE for a constant mean is the sample mean (the pika example from Monday's lecture). A difference of 20.9 NLL units is a likelihood ratio of about $e^{20.9} \approx 10^9$. Most students should find this convincing. Good answers also note a caution: the road model has one more parameter, so it would fit at least somewhat better even if the road did nothing. How to account for that formally comes later in the semester.

Watch for.

- Mixing conventions: a `dpois()` NLL for one model and a constant-dropped NLL for the other (difference of about 155, meaningless).

- Saying the smaller NLL model is “proven” or “correct.” The road model is better supported (corroborated), not proven.
- Using the median or another summary for $\hat{\lambda}$.

2e.

Path A: write out the prompt you used. Did the AI’s approach match your plan from 1e? What did you have to check or fix? **Path B:** what was hardest to translate from your plan into code?

Sample points (no fixed answer).

- **Path A.** Strong prompts name the model, the distribution, the data vectors, and ask for an NLL function plus `optim()`. Look for students who checked the AI’s code against the 2a value, rather than trusting it. Common AI divergences: using `nls()` or least squares instead of a likelihood, or using `glm()`, which can’t fit this curve directly.
- **Path B.** Common answers: getting `dpois()` arguments in the right order, passing extra arguments (`y`, `d`) through `optim()`, and understanding the warnings.

If you finish early.

A colleague prefers a different saturating curve, $\lambda(d) = \frac{ad}{b+d}$. Fit it the same way and compare its NLL with the monomolecular model’s. What do you conclude?

Answer.

```
nll_mm <- function(par, y, d) {
  -sum(dpois(y, lambda = par[1] * d / (par[2] + d), log = TRUE))
}
fit_mm <- optim(c(15, 1), nll_mm, y = males, d = dist_km)
round(c(a = fit_mm$par[1], b = fit_mm$par[2], nll = fit_mm$value,
        diff = fit_mm$value - fit$value), 3)
```

```
##      a      b    nll    diff
## 16.136 1.918 33.186 0.223
```

The two NLLs differ by only about 0.22 units. These data can’t tell the two curves apart.

Why (discussion). Both curves rise and level off, and both have two parameters. They differ mostly beyond 8 km (the Michaelis-Menten curve approaches its asymptote more slowly, here $\hat{a} \approx 16.1$), where there are no data. Choosing between them would take more ponds far from the road, or a biological reason to prefer one shape.

Watch for. Declaring the monomolecular model the winner because its NLL is slightly lower.

Reflection

In Act 1, a single visit detected this frog only about 40% of the time. What does that suggest about what the counts in this Act really measure? Also record any changes you made, new insights, and any “muddiest points” or topics you want to review.

Discussion points. The counts measure males *heard*, not males present. The expected count is roughly (number present) $\times$ (probability each is heard), so a is an index of chorus size, not abundance. The key insight for strong answers: traffic noise could lower the count near the road by masking calls (lower detection), by reducing how many males call, or by reducing how many frogs live there. These data can’t separate those explanations. Separating them would take repeat visits with a detection model like Act 1’s.

Muddiest points worth noting for the next lecture: why the constant can be dropped, what the warnings mean, and how big an NLL difference has to be to matter.